

A Precision-Controlled Tanh Construction: Arithmetic Contract and Implementation Evidence

Section 3 source note 1 — 25 September 2026

Abstract

The expC11 computation constructs a fixed-geometry tanh network and fits its readout with an explicitly rounded, p -significant-bit implementation of reference LAPACK’s SVD least-squares driver. The same arithmetic controls geometry, activation evaluation, the solve, stored coefficients, and sequential inference. This note specifies that computation, including its input boundary, exponent range, operation order, and rank cutoff. It records the existing implementation checks and completed chirp and Runge measurements, without claiming a new test run or a certificate for a different recovery algorithm.

1 What the experiment measures

The question is whether a prescribed tanh geometry can be constructed, fitted, and evaluated accurately when the numerical computation itself is restricted to p significant binary digits. Rounding a final coefficient vector is not enough to answer this question: feature formation and the least-squares solve can themselves introduce errors. ExpC11 therefore fixes a sequence of rounded operations for all of these stages.

The objects entering the experiment are a set of target samples, a bandwidth λ , integer dimensions, and a floating-point format. Target sampling and bandwidth selection occur outside the controlled computation. They produce data that are rounded before entering it. Within the computation, the centers, features, least-squares solution, and predictions are all formed in the chosen format. A binary64 observer measures the resulting error afterwards.

This distinction matters when connecting the experiment to Section 3’s realizability discussion. The experiment supplies an explicit implementation and numerical evidence for one construction procedure. It is not a proof that every solver, summation order, or parameterization has the same error, and it does not implement the supplied reference-scaled FP64 recovery theorem. That theorem has its own coefficient-envelope scaling and recovery/export contract. ExpC11 uses the unscaled design matrix and the particular truncated DGELSS procedure described below.

The earlier expC09 precision panel also has a different interpretation: its precision restriction concerns the forward computation while the solve remains in FP64. Its observed plateau and error constant cannot be presented as those of an all- p -bit solve. The later expC13 raw-affine comparison is likewise a separate protocol. None of its measurements are used in this note.

2 The arithmetic contract

2.1 Significand precision and exponent range

Let $\mathcal{F}(p, e_{\min}, e_{\max})$ denote a binary format with p significand bits, including the leading bit, normal exponents from $e_{\min}$ through $e_{\max}$, and gradual underflow. A positive normal value has the form $m2^e$ with $1 \leq m < 2$ and p binary significant digits. Subnormals below $2^{e_{\min}}$ lie on the uniform lattice with spacing

$$q_{\text{sub}} = 2^{e_{\min}-p+1}.$$

Write $\mathcal{Q}$ for round-to-nearest, ties-to-even into this format. The five arithmetic primitives are

$$\begin{aligned} \text{add}(a, b) &= \mathcal{Q}(a + b), & \text{sub}(a, b) &= \mathcal{Q}(a - b), \\ \text{mul}(a, b) &= \mathcal{Q}(ab), & \text{div}(a, b) &= \mathcal{Q}(a/b), & \text{sqrt}_p(a) &= \mathcal{Q}(\sqrt{a}). \end{aligned}$$

Every operand is already a format value, and each primitive rounds separately. In particular, a multiply followed by an add is not fused. The unit roundoff for normal results is $u_p = 2^{-p}$; the primary solver cutoff is $2u_p$.

Negation, absolute value, comparison, sign transfer, and copying do not require a rounding step. Power-of-two scaling is exact when its result is representable; the emulator’s scaling wrapper also applies format rounding at range boundaries. The nearest-integer operation `rint`, with ties to even, is used only on the small values required by activation range reduction. An integer used as a floating operand enters as $\mathcal{Q}(k)$. Integer loop counters, array sizes, and indexing remain ordinary exact control information.

The precision sweep uses

$$\mathcal{F}_p = \mathcal{F}(p, -958, 959), \quad p = 8, 9, \dots, 53.$$

This range is wide enough to separate the experiment’s significand restriction from the narrow exponent range of IEEE binary32. It also ensures that every finite sweep-format value, including its subnormals, is exactly representable as a normal binary64 carrier value. Native-format checks use

$$\mathcal{F}_{32} = \mathcal{F}(24, -126, 127), \quad \mathcal{F}_{64} = \mathcal{F}(53, -1022, 1023).$$

Thus the sweep point $p = 24$ is not an IEEE FP32 experiment. The sweep format at $p = 53$ is also not identical to binary64 as a set of numbers. The recorded agreement with binary64 concerns the tested computations; exponent-sensitive behavior need not agree on arbitrary inputs.

2.2 Constants and the input boundary

Constants are prepared outside the model and rounded once into the format. This includes the activation coefficients and the machine parameters used by LAPACK. The preparation code uses exact rational arithmetic for rational constants and high-precision evaluation with a midpoint-separation check for the required transcendental constants. Runtime access retrieves the stored format value; it does not recompute that constant at higher precision.

For binary64 training locations x_i and supplied binary64 labels $y_i = \text{fl}_{64}(f(x_i))$, the model receives

$$\tilde{x}_i = \mathcal{Q}(x_i), \quad \tilde{y}_i = \mathcal{Q}(y_i).$$

It likewise receives rounded evaluation inputs $\tilde{x}_i^e$, bandwidth $\tilde{\lambda} = \mathcal{Q}(\lambda)$, and geometry integers $\tilde{N} = \mathcal{Q}(N)$ and $\tilde{j} = \mathcal{Q}(j)$. The labels are samples at the original locations x_i , subsequently rounded; they are not freshly evaluated as $f(\tilde{x}_i)$ inside the model.

The bandwidth is selected offline, separately for each precision, using the existing refined rule with tolerance 2^{1-p} . Consequently, the experiment does not assert that target generation or the bandwidth-selection algorithm itself runs at p bits. Its controlled boundary begins after those inputs have been supplied and rounded.

3 Geometry, activation, and design matrix

3.1 Centered features in a prescribed operation order

Choose N interior intervals and H halo centers on each side. The center indices are $j = -H, \dots, N+H$, giving

$$W = N + 2H + 1$$

hidden features. Geometry is formed by

$$h = \text{div}(2, \tilde{N}), \quad c_j = \text{add}(-1, \text{mul}(\tilde{j}, h)), \quad \gamma = \text{div}(\tilde{\lambda}, h). \quad (1)$$

The feature at a format-valued input x is

$$\phi_j(x) = \tanh_p(\text{mul}(\gamma, \text{sub}(x, c_j))). \quad (2)$$

The subtraction is performed before multiplication by γ . Forming a stored bias $b_j = \mathcal{Q}(-\gamma c_j)$ and evaluating $\text{add}(\text{mul}(\gamma, x), b_j)$ gives a different rounding sequence, even though the two expressions are identical over the reals. The expC11 evidence applies to the centered form (2).

At small p , the rounding of geometry integers and centers is part of the experiment. Distinct indices can produce equal rounded values, and a fine training grid can have repeated rounded locations. The array still has the prescribed number of rows and columns. One should not interpret that nominal width as a guarantee of distinct finite-precision features or full rank.

3.2 A tanh algorithm built from rounded primitives

The activation is a specified algorithm $\tanh_p$, not a call to a system `tanh` routine followed by output rounding. Its construction uses

$$\tanh(a) = -\frac{\exp(-2a) - 1}{\exp(-2a) + 1}, \quad a \geq 0,$$

and evaluates the numerator as an `expm1`-type expression to avoid subtracting two nearly equal numbers near the origin.

The stored constants are

$$\begin{aligned} I &= \mathcal{Q}(1/\ln 2), & S &= \mathcal{Q}((p+3)\ln 2/2), \\ L_h &= \mathcal{Q}_{\max(p-k_b, 1)}(\ln 2), & L_t &= \mathcal{Q}(\ln 2 - L_h), \end{aligned}$$

where k_b is the bit length of $p+4$ and $\mathcal{Q}_q$ denotes rounding to q significand bits. The shorter significand of L_h supports the split range reduction. Let $d \geq 1$ be the smallest integer satisfying

$$\frac{0.35^d}{(d+1)!} \leq 2^{-(p+1)}, \quad a_\ell = \mathcal{Q}\left(\frac{1}{(\ell+1)!}\right), \quad 0 \leq \ell < d.$$

These coefficients define a Horner approximation to $\exp(r) - 1$ after reduction. The degree rule is part of the specified algorithm; by itself it is not a global floating-point error bound for the completed activation.

For input z , set $a = |z|$. If $a \geq S$, return 1 or -1 according to the sign of z . Otherwise form $u = -2a$ by exact power-of-two scaling and set

$$k = \text{rint}(\text{mul}(u, I)), \quad r = \begin{cases} u, & k = 0, \\ \text{sub}(\text{sub}(u, \text{mul}(k, L_h)), \text{mul}(k, L_l)), & k \neq 0. \end{cases}$$

The small integer k is represented in the working format when used in these products. Evaluate the polynomial in the following order:

$$q \leftarrow a_{d-1}, \quad q \leftarrow \text{add}(a_\ell, \text{mul}(r, q)) \quad (\ell = d-2, d-3, \dots, 0), \quad P = \text{mul}(r, q).$$

Reconstruct the exponential difference and the activation by

$$E = \begin{cases} P, & k = 0, \\ \text{add}(\text{ldexp}(P, k), \text{sub}(2^k, 1)), & k \neq 0, \end{cases}$$

$$t = -\text{div}(E, \text{add}(E, 2)), \quad \tanh_p(z) = \begin{cases} t, & z \geq 0, \\ -t, & z < 0. \end{cases}$$

Every displayed add, subtract, multiply, and divide is one rounded operation. There is no wider accumulator inside the activation algorithm. Nor is $\tanh_p(z)$ claimed to equal correctly rounded mathematical $\tanh(z)$. The existing tests compare the algorithm with an independent MPFR replay and check its error against mathematical $\tanh$ on sampled inputs; these are different checks.

3.3 The sampled fitting problem

For M training points, form the $M \times n$ matrix, $n = W + 1$,

$$A_{i,j+H+1} = \phi_j(\tilde{x}_i), \quad A_{i,n} = 1, \quad -H \leq j \leq N + H.$$

The final column represents the output bias. The right-hand side is $\tilde{y} = (\tilde{y}_1, \dots, \tilde{y}_M)^T$. Entries of A are already format values when the least-squares driver receives them. The driver performs its own rounded matrix operations; it does not hand this matrix to a binary64 solver.

4 The all-p-bit least-squares computation

4.1 Reference driver and supported branch

The implementation ports reference LAPACK 3.12.1 DGEELSS, restricted to $M \geq n$ and one right-hand side. Floating-point expressions in the reachable routines are mapped to the primitives above in the specified evaluation order. Block size is fixed at one, selecting the unblocked level-2 kernels. A blocked library implementation can change rounding order and is not the bitwise reference for this computation.

For the tall expC11 matrix, the main stages are Householder QR, application of its orthogonal transformations to the right-hand side, reduction of the remaining problem to upper bidiagonal form, construction of the right singular-vector factors, implicit-shift QR iteration on the bidiagonal matrix, and truncated back-substitution. The port includes the scalar operations inside these stages: norms, rotations, reflectors, matrix-vector products, and scaling. Representative routines are DGEQR2, DLARFG, Blue's DNRM2, DGEED2, DORGL2, DBDSQR, DLARTG, DLAS2, DLASV2, DRSCL, and DGEMV.

Machine parameters belong to the working format. In particular, `DLAMCH('E')` is 2^{-p} and `DLAMCH('P')` is 2^{1-p} . Safe minimum, overflow threshold, norm-scaling thresholds, and the constants used by bidiagonal iteration are also prepared for that format. Changing only the rounding of matrix entries while retaining binary64 machine constants would define a different solver.

The port is deliberately not a general replacement for all `DGELSS` branches. It returns an error for $M < n$, an input requiring the omitted `DLASCL` rescaling branch, or an all-zero matrix. A constant bias column rules out the all-zero design matrix here. A nonzero bidiagonal-iteration status is propagated rather than silently accepted as a fitted model.

There is one documented arithmetic exception in solver control flow. The reference driver's dimension crossover is computed as

$$\text{MNTHR} = \text{INT}(\text{REAL}(\min(M, n)) \cdot 1.6)$$

in native single precision, matching reference LAPACK. It determines whether to perform the initial QR reduction. This calculation uses matrix dimensions, not data values; it is outside the p -bit numerical kernels and is explicitly allowed by the static audit. For the reported 4801×1025 problem it selects the tall-matrix QR path.

4.2 Truncation and sensitivity cutoffs

In exact arithmetic, a truncated SVD solve retains singular directions whose singular values exceed a threshold, divides their transformed right-hand-side components by those singular values, and maps back through the right singular vectors. In `expC11` each of these operations, including the decomposition, is performed in the working arithmetic. Therefore the exact-arithmetic formula describes the algorithmic purpose, not an identity asserting that the computed factors are an exact SVD of A .

Let $\hat{\sigma}_1 \geq \dots \geq \hat{\sigma}_n$ be the computed singular values, and let `sfin` be the stored LAPACK safe minimum. The implemented retention rule is

$$\hat{\sigma}_i > au, \quad \tau = \max\{\text{mul}(\rho, \hat{\sigma}_1), \text{sfin}\}, \quad \rho = 2^{1-p}. \quad (3)$$

Equality is discarded. This includes the safe-minimum safeguard present in the code; the shorter description “discard values at most $\rho\sigma_1$ ” suppresses that detail. The reported effective rank is the number of retained computed singular values.

The chirp sensitivity study also uses $\rho = \kappa 2^{-p}$ for $\kappa \in \{1, 8, 32\}$. It reuses the state after `DBDSQR` and repeats the cutoff-dependent scaling and final matrix–vector multiply. Since the decomposition does not depend on `RCOND`, this reproduces those stages of separate calls with the same arithmetic and decomposition. A cutoff is not chosen afterwards to minimize the plotted primary error: the primary curve consistently uses $\rho = 2^{1-p}$.

5 Storage, inference, and the external observer

Write the fitted coefficients as w_j , $-H \leq j \leq N + H$, and bias b . All are format values produced by the solver. For an evaluation input $\tilde{x}$, the readout starts at the bias and visits the features in increasing center-index order:

$$s \leftarrow b, \quad s \leftarrow \text{add}(s, \text{mul}(\phi_j(\tilde{x}), w_j)), \quad j = -H, -H + 1, \dots, N + H. \quad (4)$$

This is a sequential rounded sum. Pairwise reduction, compensated summation, a library dot product, or an exact accumulator would change the experiment. Saved-model evaluation reconstructs the same rounded geometry and applies this same order.

The emulator stores values in binary64 arrays, but the storage type does not grant additional arithmetic precision: each stored number must be exactly representable in the chosen format, and subsequent arithmetic still goes through its rounding primitives. Saved coefficient, prediction, and singular-value arrays are checked for this representability. The saved model files use binary64 containers; they are not a claim of a physically packed p -bit deployment format or a measured memory saving.

With M_e original evaluation locations x_i^e , define the reported error

$$E_p = \frac{\left(\sum_{i=1}^{M_e} |s(\mathcal{Q}(x_i^e)) - f_{64}(x_i^e)|^2\right)^{1/2}}{\left(\sum_{i=1}^{M_e} |f_{64}(x_i^e)|^2\right)^{1/2}}, \quad f_{64}(x) = \text{fl}_{64}(f(x)).$$

The observer performs this measurement in binary64. In particular, it compares against the target at the unrounded grid, so input quantization contributes to the error. This is a finite-grid relative ℓ^2 measurement, conventionally called relative L^2 in the experiment reports. It is not a certified continuous-domain norm, and no sampled-to-continuous theorem is invoked here.

6 Implementation checks and their scope

The model and solver are written once in two C headers and instantiated with three arithmetic backends. The emulator uses a binary64 carrier to implement the requested rounded primitives; the native variants use binary32 or binary64 scalar types. Native builds disable contraction and automatic vectorization, preserving the scalar operation sequence.

The emulator must avoid double rounding. A binary64 operation followed by a naive p -bit rounding can choose the wrong side of a target-format midpoint. The implementation therefore computes the carrier result together with the sign of its residual relative to the exact result. TwoSum supplies this information for addition; FMA residuals supply it for multiplication, division, and square root. The residual sign resolves midpoint cases before the result is returned in the target format. Exact power-of-two scaling protects very small residual calculations. The internal use of FMA implements one rounded primitive; it does not fuse a model multiply with its following model add.

The completed reports record 202 passing tests in the arithmetic and replay suite, together with full-size native/reference comparisons. These are historical results from those reports, not tests rerun in preparing this note. The evidence has several distinct roles:

1. Primitive comparisons use MPFR for the rounded add, subtract, multiply, divide, and square-root operations, including midpoint traps, subnormal behavior, and tiny results. Native-format comparisons additionally check agreement with hardware arithmetic.
2. A separate MPFR implementation, written from the specification, replays activation constants, $\tanh_p$, feature formation, and sequential readout. It does not independently replay the least-squares solve. Its model replay uses a wide exponent range without subnormalization; finite-range behavior is covered separately by the primitive tests.
3. The solver is compared with netlib SGELSS/DGELSS, compiled from reference source with contraction disabled and block size one. The tests cover random, rank-deficient, graded, and tanh design matrices. The reports also record bitwise agreement on the full 4801×1025 problem in exact IEEE FP32 and FP64.

4. The static audit walks clang’s expanded syntax tree, including every branch, and rejects raw floating arithmetic, unrounded conversions, disallowed math calls, or unchecked floating literals outside the rounding layer. Its allowed findings are exactly the dimension-crossover exception described above. Planted arithmetic and conversion leaks serve as negative controls.
5. The Runge report records representability and error-recomputation checks for all 46 saved models, source-hash agreement with its manifest, and bitwise saved-model replay at $p = 8, 24, 53$.

These checks support fidelity to a specified numerical algorithm. They do not constitute an exhaustive proof over all inputs, a certified tanh error bound, or a conditioning theorem. In particular, the existing tanh test samples inputs at each precision from 8 through 53 and requires error at most three ulps, where the ulp is measured at the true output’s magnitude. The report’s stronger phrasing “at most 2.5 ulp at every p ” should be read as reported numerical evidence, not a uniform bound proved by that test.

7 Completed measurements

The two reported sweeps use $W = 1024$, $H = 24$, and $N = 975$, with 4801 training points and 8001 evaluation points, equispaced on $[-1, 1]$. The targets are

$$f_{\text{chirp}}(x) = \sin(8\pi(x+1)^2), \quad f_{\text{Runge}}(x) = \frac{1}{1+25x^2}.$$

The offline selector uses frequency scales 16π and 5, respectively. For Runge, the search interval is $[0.03, 3]$: at $p = 8, 9, 10, 11$ the selector returns its upper limit $\lambda = 3$, rather than an equality root. At $p = 12, \dots, 53$ the report records satisfaction of the stated score equation.

Table 1 reproduces rounded values from the completed reports. Both main curves use the sweep exponent range and cutoff 2^{1-p} .

Table 1: Reported relative evaluation errors. The IEEE FP32 anchor has a different exponent range from the 24-bit sweep point.

Format	Chirp	Runge 25
Sweep, $p = 24$	6.81×10^{-5}	9.57×10^{-6}
Sweep, $p = 53$	1.04×10^{-13}	2.50×10^{-14}
IEEE binary32 anchor	6.39×10^{-5}	not reported here

For chirp, a line $E_p \approx C2^{-p}$ with slope fixed in advance was fit by its intercept over $p = 16, \dots, 40$, giving $C = 1386$. Over $p = 16, \dots, 53$, the reported median of $E_p 2^p$ is 1115. The sweep continues down to $p = 53$ without the separate plateau visible in the forward-only comparison. The latter’s reported fitted constant is 19.5, and its FP64-solve plateau is approximately 9.3×10^{-14} . The data support a one-bit-per-bit trend for this fixed-size, fixed-solver experiment; they do not identify a universal error constant for least squares.

Low precision is a distinct regime. For chirp at $p = 8, \dots, 13$, the reported errors are 1.35, 1.46, 0.84, 0.39, 0.29, 0.25, with retained ranks 100, 232, 299, 575, 871, 943. Larger sensitivity cutoffs reduce several spikes in the higher-precision curve. Thus the trend must not be described as monotone convergence at every precision or as successful accurate fitting throughout the 8–53 range.

Runge generally has lower error through most of the sweep, including the 24-bit and 53-bit entries in the table. No convergence-rate or intercept fit was performed for Runge. Its final two errors, 2.66×10^{-14} at $p = 52$ and 2.50×10^{-14} at $p = 53$, do not alone establish a floor.

The chirp IEEE FP32 anchor also illustrates why the exponent range must be reported. Its error differs by about six percent from the 24-bit sweep point; the report records approximately 1.8×10^8 subnormal intermediates in that FP32 run. This is an observed effect of the full format, not evidence that the significand-bit specification was violated. Both source reports retain a pending-author-review status for presentation or conclusions, even though the cited measurement sweeps are complete.

8 Source provenance and reproducibility boundary

The following are actual paths relative to the repository root. They separate the computation’s specification, inspected implementation, historical test evidence, and saved experimental results. No training run, precision sweep, or numerical validation suite was rerun for this note.

Normative contract. `experiments/expC11_true_precision_law/SPEC.md`. The explicit safe-minimum threshold in (3), zero-matrix rejection, and dimension-crossover exception are clarified from the code.

Model and solver. `src/precision/pbit_algo.h` and `src/precision/lapack_gelss.h`.

Arithmetic and interface. `src/precision/pbit_emul.c`, `src/precision/pbit_native.c`, and `src/precision/pbit.py`.

Independent anchors and audit. `src/precision/reference_lapack.py` and `src/precision/audit.py`.

Tests and model replay. `tests/test_pbit.py`, `tests/pbit_replay_reference.py`, and `tests/test_pbit_replay_selfcheck.py`.

Experiment runners. `experiments/expC11_true_precision_law/run.py` and `experiments/expC11_true_precision_law/runge_comparison.py`.

Chirp report. `results/checkpoint_C_geometry/expC11_true_precision_law/expC11_results.md`. Its sibling `data/` directory contains `config.json`, `measurements.jsonl`, `summary.json`, the saved models/`p*.npz`, and `native/reference` anchor records.

Runge report. `results/checkpoint_C_geometry/expC11_true_precision_law/runge25_comparison/expC11_runge25_results.md`. Its sibling `data/` directory contains the configuration and source hashes, `measurements`, `models`, and `validation.json`.

Distinct recovery proof supplied with the source packet. `docs/appendix_materials/2026-09-25/supplied/fp64_scaled_svd_recovery_proof.pdf`. It is identified here to keep its numerical contract separate; the expC11 checks are not a certificate that this theorem’s algorithm was implemented.